

Lecture 16

Friend Function and Friend Classes

Object Oriented
Programming

friend Functions and friend Classes

- So far we have seen that the private members of a class can only be accessed using the public `getter()/setter()` function of that class.
- However, it is possible to access private members of a class without using public getter function, by granting friendship to another function or class.
- **friend** function and **friend** classes
 - Can access **private** and **protected** members of another class
 - **friend** functions are not member functions of class
 - Defined outside of class scope
- Friendship is granted, not taken

friend Function declaration

- **friend** declarations
 - To declare a **friend** function
 - Type keyword **friend** before the function prototype in the class that is giving friendship

```
friend int myFunction( int x );
```

should appear in the class giving friendship

- It does not matter whether you declare the friend function in the private or public section of the class.

Friend non-member Function: Example

```
class Rectangle {  
    int width, height;  
public:  
    Rectangle(int w=1, int h=1):  
        width(w), height(h) {}  
    friend void display(Rectangle &);  
};  
  
void display(Rectangle &r) {  
    cout << r.width * r.height << endl;  
}
```

```
int main () {  
    Rectangle rect(5,10);  
    display(rect);  
    return 0;  
}
```

- Note that we don't need Rectangle's object to call display().
- Display() is not a member function of Rectangle.
- Inside display() private members of Rectangle can be accessed directly.

Multiple Friends: Another Example

```
class Square; // forward declaration
class Rectangle {
    int width, height;

public:
    Rectangle(int w=1, int h=1):
        width(w), height(h) {}

    friend void display(Rectangle &, Square &);
};

class Square {
    int side;

public:
    Square(int s = 1):side(s) {}

    friend void display(Rectangle &, Square &);
};

void display(Rectangle &r, Square &s) {
    cout << "Rectangle: " << r.width * r.height
    << endl;

    cout << "Square: " << s.side * s.side <<
    endl;
}
```

• Multiple friends

- A function can be a friend of more than one class at the same time.
- Example like this is very similar to cases we'll encounter later when we discuss operator overloading!

```
int main () {
    Rectangle rec(5,10);
    Square sq(5);
    display(rec,sq);
    return 0;
}
```

Friend Member Functions

- Sometime, one class needs access to the private data members of another class.
- These private members of a class can be accessed by another class if the second class grants friendship by making a member function of the first class a Friend.

Friend Member Functions: Example

```
class Square; // forward declaration
class Display {
public:
//only prototype here. Definition later.
void displaydimension(Square &);
};

class Square {
int side;
public:
Square(int s = 1):side(s){}
friend void
display::displaydimension(Square &);
};

void Display::displaydimension(Square &s) {
cout << "Square Dimension is: " << s.side;
}
```

• Friend Member Function

- Display class needs access to the Square class private members.
- Square class grants friendship

```
int main () {
Square sq(5);
Display D;
D.displaydimension(sq);
return 0;
}
```

Friend Classes Declaration

- Just like functions are made friends of classes, we can also make one class to be a friend of another class.
 - Then, the friend class will have access to all the private members of the other class.
- To declare a **friend** class
 - Type **friend class Classname** in the class that is giving friendship
 - if **ClassOne** is granting friendship to **ClassTwo**,

```
friend class ClassTwo;
```

should appear in **ClassOne**'s definition
 - It does not matter whether you declare the friend function in the private or public section of the class.

Friend Classes: Another Example

```
class Square;

class Rectangle {
int width, height;
public:
Rectangle(int w=1, int h=1):
width(w),height(h){}
void display() {
cout << "Rectangle: " << width *
height;
}
void modifyRectangle(Square &);
};

class Square {
int side;
public:
Square(int s=1):side(s){}
void display() {
cout<< "Square:"<<side*side; }
friend class Rectangle;
};
```

```
void Rectangle::modifyRectangle(Square
&s) {
width = s.side;//accessing square
height = s.side;
}

int main () {
    Rectangle rec(5,10);
    Square sq(5);
    cout << "Before:" << endl;
    rec.display();
    sq.display();

    rec.modifyRectangle(sq);
    cout << "\nAfter:" << endl;
    rec.display();
    sq.display();
    return 0;
}
```

```
Before:
Rectangle: 50
Square: 25
```

```
After:
Rectangle: 25
Square: 25
```

Friend Classes

- In the program given on next slide, all member functions of *class B* will be friend functions of *class A*. Thus, any member function of class B can access the private and protected data of class A.
- However, member functions of class A cannot access the data of class B.
- Remember, friend relation in C++ is only granted, not taken.

```
class ClassB;

class ClassA {
    private:
        int numA;

        // friend class declaration
        friend class ClassB;

    public:
        // constructor to initialize numA to 12
        ClassA() : numA(12) {}
};

class ClassB {
    private:
        int numB;

    public:
        // constructor to initialize numB to 1
        ClassB() : numB(1) {}

        // member function to add numA
        // from ClassA and numB from ClassB
        int add() {
            ClassA objectA;
            return objectA.numA + numB;
        }
};
```

```
int main() {
    ClassB objectB;
    cout << "Sum: " << objectB.add();
    return 0;
}
```

Class exercise

Suppose there is a class named Complex, it has two data members of type float, real and imag. Of course it has a constructor as well. in main we create two object a and b. Using a friend function named add(), we pass a and b which adds the corresponding data members of a and b and returns a new complex object to the main().

Friendship is not inherited

- Just because I grant you friendship access to me doesn't automatically grant your kids access to me, doesn't automatically grant your friends access to me, and doesn't automatically grant me access to you.
- I don't necessarily trust the kids of my friends. The privileges of friendship aren't inherited. Derived classes of a friend aren't necessarily friends.
- If class **Rectangle** declares that class Base is a friend, classes derived from Base don't have any automatic special access rights to **Rectangle** objects.

Friendship is not transitive

- I don't necessarily trust the friends of my friends.
- The privileges of friendship aren't transitive.
- A friend of a friend isn't necessarily a friend.
- If class **Rectangle** declares class **Square** as a friend, and class **Square** declares class **Triangle** as a friend, class **Triangle** doesn't necessarily have any special access rights to **Rectangle** objects.

Friendship is not reciprocal

- You don't necessarily trust me simply because I declare you my friend.
- The privileges of friendship aren't reciprocal.
- If class **Rectangle** declares that class **Square** is a friend, **Square** objects have special access to **Rectangle** objects but **Rectangle** objects do not automatically have special access to **Square** objects.